The Django layer

Study guide 5: models, forms, views, URLs, admin and settings around the pandas core

What this guide covers

File	Job
`core/models.py`	ReportRun, the abstract base model, and the upload folders
`commission/models.py (activity/models.py is the same shape)`	CommissionRun: one saved report
`core/forms.py, commission/forms.py`	Checking the month, rate and upload before any work
`commission/views.py (activity/views.py is the same shape)`	List, create, detail, download
`commission/urls.py, nexora_reports/urls.py`	Addresses to views
`commission/admin.py`	Runs in the Django admin
`nexora_reports/settings.py (the parts we changed)`	Apps, templates, media, login, ledger

How to use it

- Read Part 2's three pictures first: the address-to-view path, the create view's branches, and the model family.
- Rebuild commission's Django files in Part 3's order: model, form, views, URLs, admin, settings.
- Run the checkpoints in Part 5; they were extracted from this guide and pass against the project.

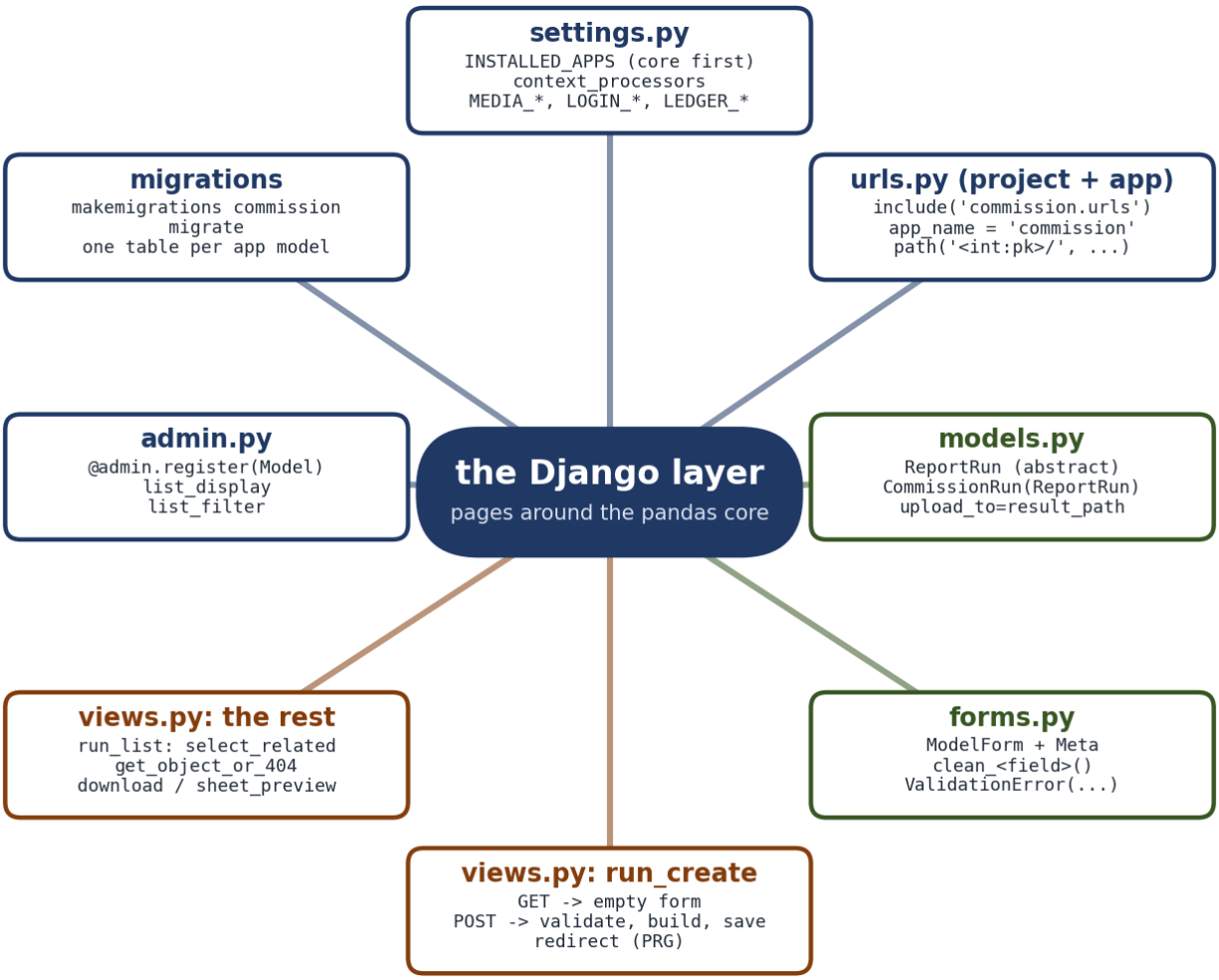
Part 1. Think before you write code

The Django files don't calculate anything. Their job is to get a valid request to `services.build_commission`, save what it returns, and show it. So the questions are about **flow**, not formulas:

1. **Which address runs which code?** URLs.
2. **What does the user send, and is it valid?** Forms.
3. **What must be remembered after the request ends?** Models.
4. **What happens, in what order, on success and on failure?** Views.
5. **How do the pieces find each other?** Settings.

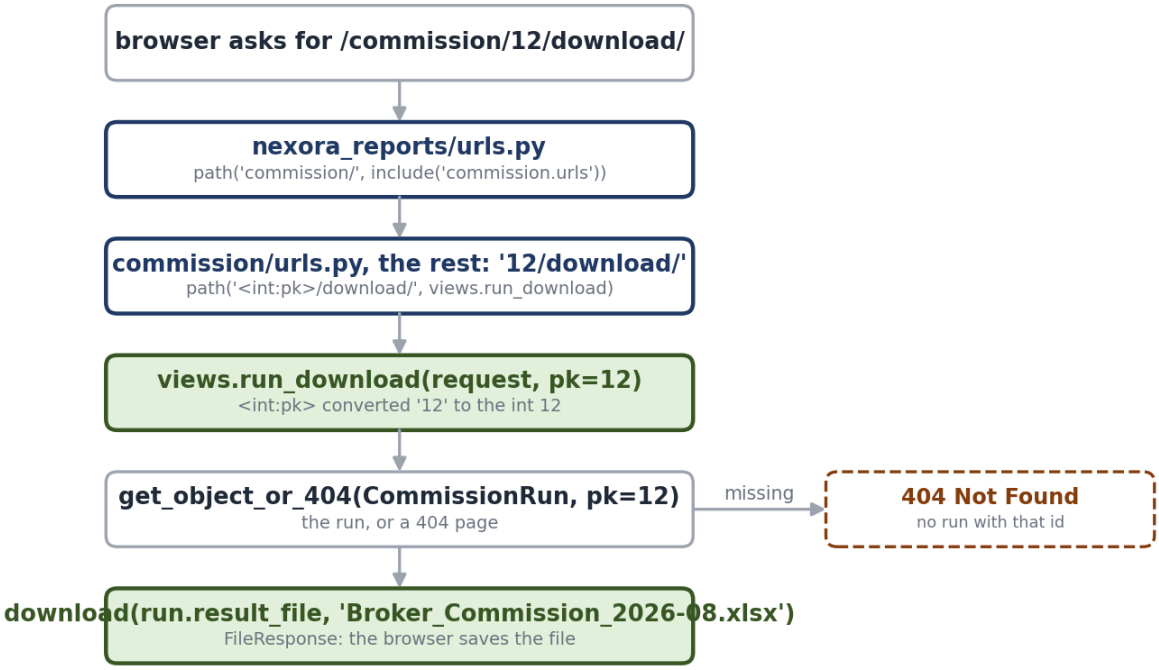
Fact	So the code...	Piece
Every report run has a month, a file, an author and a time	puts those fields in one base	`ReportRun (abstract)`
Several apps store files	files each app under its own folder	`upload_to=result_path`
Bad input should be explained, not crash	validates in the form, then catches ValueError	`clean_<field>, form.add_error`
Refreshing a results page must not rerun the report	redirects after a successful POST	`redirect('...:run_detail')`
Commission data is internal	requires login on every view	`@login_required`
Money must be exact	stores rates and totals as decimals	`DecimalField, Decimal(str(x))`

The Django layer as a mindmap

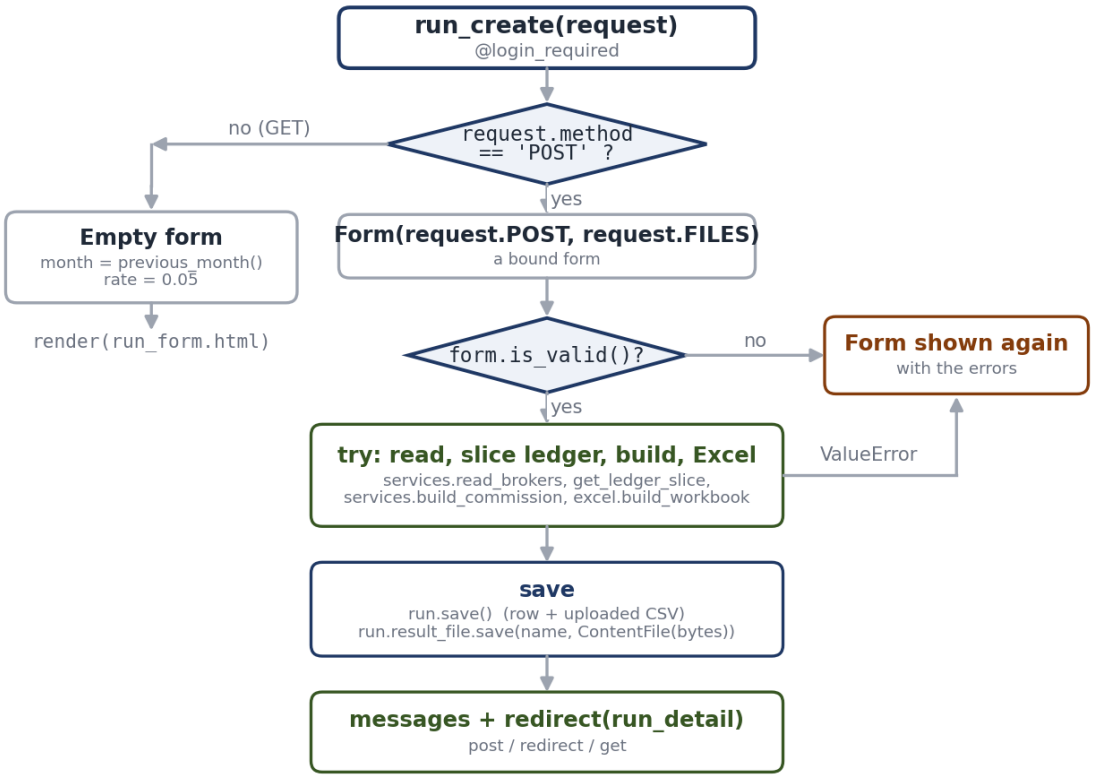

settings.py
INSTALLED_APPS (core first)
context_processors
MEDIA_*, LOGIN_*, LEDGER_*
migrations
makemigrations commission
migrate
one table per app model
urls.py (project + app)
include('commission.urls')
app_name = 'commission'
path('<int:pk>/', ...)
admin.py
@admin.register(Model)
list_display
list_filter
the Django layer
pages around the pandas core
models.py
ReportRun (abstract)
CommissionRun(ReportRun)
upload_to=result_path
views.py: the rest
run_list: select_related
get_object_or_404
download / sheet_preview
forms.py
ModelForm + Meta
clean_<field>()
ValidationError(...)
views.py: run_create
GET -> empty form
POST -> validate, build, save
redirect (PRG)

Part 2. The big picture

From an address to a view

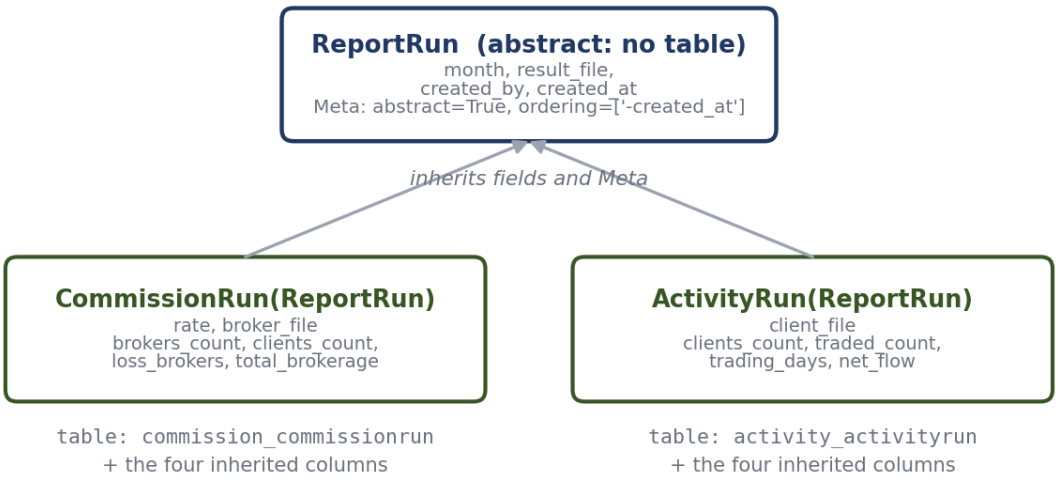


What run_create does on each path



Every path that doesn't succeed ends with the same form template; success always ends in a redirect.

The model family



Part 3. Build it step by step

Step 1. ReportRun: the fields every report shares

What I was thinking
"Every report needs a month, a result file, who ran it and when. Writing those in each app's model would repeat them; an abstract base model holds them once."
"abstract = True: Django creates no table for ReportRun itself; each child gets these columns in its own table."
"Files: each app in its own folder, results grouped by year and month. upload_to can be a function that builds the path from the object."

```
from django.conf import settings
from django.db import models


def result_path(instance, filename):
    """Each app's results go in their own folder: media/<app>/results/2026/11/<file>."""
    return f"{instance._meta.app_label}/results/{instance.created_at:%Y/%m}/{filename}"


def input_path(instance, filename):
    return f"{instance._meta.app_label}/inputs/{filename}"


class ReportRun(models.Model):
    """Fields every saved report has. Abstract: no table of its own; each app's run model adds its fields."""

    month = models.CharField(max_length=7, help_text="YYYY-MM, e.g. 2026-08")
    result_file = models.FileField(upload_to=result_path, blank=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
                                   on_delete=models.SET_NULL, related_name="+")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        abstract = True
        ordering = ["-created_at"]
```

Tool	What it does	Why here
`class Meta: abstract = True`	A base with no table	Children inherit fields and Meta
`ordering = ['-created_at']`	Default sort, newest first	Every list is newest first without asking
`upload_to=result_path`	A function that names the stored file	media/commission/results/2026/11/...
`instance._meta.app_label`	The app the model belongs to	'commission' or 'activity'
`f'{instance.created_at:%Y/%m}'`	Formats a datetime inside an f-string	2026/11
`ForeignKey(settings.AUTH_USER_MODEL, ...)`	A link to a user	Works with Django's user or a custom one
`on_delete=models.SET_NULL, null=True`	Deleting a user keeps their runs	The run's author becomes empty
`related_name='+'`	No reverse accessor on User	No clashes between apps' run models
`auto_now_add=True`	Set once, when the row is created	The run's timestamp

Check yourself
CommissionRun.ordering is ['-created_at'] although commission/models.py never mentions ordering.

Why the view saves in a fixed order
created_at is filled in only when the run is first saved. Before that it's None, and result_path fails with `TypeError: unsupported format string passed to NoneType.__format__` (checked by running it). So the view calls `run.save()` first and `run.result_file.save(...)` second.

Step 2. CommissionRun: the app's own fields

What I was thinking
"Inherit the four shared fields; add what only this report needs: the rate, the uploaded file and the numbers to show in the list."
"Money and rates are DecimalField: floats store 0.1 approximately, decimals store it exactly."
"__str__ gives a readable name in the admin and in error messages."

```
from decimal import Decimal

from django.db import models

from core.models import ReportRun, input_path


class CommissionRun(ReportRun):
    """One broker commission report. month, result_file, created_by and created_at come from ReportRun."""

    rate = models.DecimalField(max_digits=5, decimal_places=4, default=Decimal("0.05"))
    broker_file = models.FileField(upload_to=input_path)

    brokers_count = models.PositiveIntegerField(default=0)
    clients_count = models.PositiveIntegerField(default=0)
    loss_brokers = models.PositiveIntegerField(default=0)
    total_brokerage = models.DecimalField(max_digits=14, decimal_places=2, default=Decimal("0"))

    def __str__(self):
        return f"Commission {self.month} ({self.created_at:%Y-%m-%d %H:%M})"


# then, in a terminal:
#   python manage.py makemigrations commission
#   python manage.py migrate
```

Tool	What it does	Why here
`class CommissionRun(ReportRun)`	Inherits the base's fields	Seven fields of its own + four shared
`DecimalField(max_digits=, decimal_places=)`	An exact decimal number	0.0500, 9784.60
`default=Decimal('0.05')`	Built from text	Decimal(0.05) would carry the float's error
`PositiveIntegerField(default=0)`	A whole number, never negative	Counts
`def __str__(self)`	How the object prints	Shown in the admin and the shell
`makemigrations / migrate`	Writes the change script / applies it	Creates commission_commissionrun

Check yourself
After migrate, python manage.py dbshell (or the admin) shows the table with eleven columns plus id.

Step 3. The forms: check everything before any work

> **What I was thinking**
> "The month picker sends '2026-08', but anyone can post anything. A regular expression checks the exact shape."
> "A ModelForm builds fields from the model; Meta says which fields the user may fill. The counts and created_by are never user input."
> "clean_ runs after Django's basic checks; raising ValidationError puts my message next to that field. Shared checks live in core.forms."

```
# core/forms.py
import re
from django import forms

MAX_UPLOAD_MB = 5


def clean_month_value(month: str) -> str:
    if not re.fullmatch(r"\d{4}-(0[1-9]|1[0-2])", month or ""):
        raise forms.ValidationError("Use the format YYYY-MM, e.g. 2026-08.")
    return month


def clean_csv_upload(f, max_mb: int = MAX_UPLOAD_MB):
    if not f.name.lower().endswith(".csv"):
        raise forms.ValidationError("Upload the file as a .csv.")
    if f.size > max_mb * 1024 * 1024:
        raise forms.ValidationError(f"The file is larger than {max_mb} MB.")
    return f


MONTH_WIDGET = forms.TextInput(attrs={"type": "month"})       # the browser's month picker


# commission/forms.py
class CommissionRunForm(forms.ModelForm):
    class Meta:
        model = CommissionRun
        fields = ["month", "rate", "broker_file"]
        widgets = {"month": MONTH_WIDGET,
                   "rate": forms.NumberInput(attrs={"step": "0.005", "min": "0", "max": "1"})}
        labels = {"rate": "Commission rate (0.05 = 5%)", "broker_file": "Broker referrals CSV"}

    def clean_month(self):
        return clean_month_value(self.cleaned_data["month"])

    def clean_rate(self):
        rate = self.cleaned_data["rate"]
        if not Decimal("0") < rate <= Decimal("1"):
            raise forms.ValidationError("The rate must be above 0 and at most 1 (0.05 means 5%).")
        return rate

    def clean_broker_file(self):
        return clean_csv_upload(self.cleaned_data["broker_file"])
```

Tool	What it does	Why here
`re.fullmatch(pattern, text)`	True only if the WHOLE text matches	'2026-08x' is rejected
`\d{4}-(0[1-9]|1[0-2])`	Four digits, a dash, month 01 to 12	'2026-13' is rejected
`month or ''`	Empty value becomes ''	fullmatch never receives None
`forms.ValidationError(msg)`	A field error with your message	Shown next to the field
`f.size > max_mb * 1024 * 1024`	Bytes compared with megabytes	Refuses huge uploads early
`class Meta: model, fields, widgets, labels`	How the ModelForm builds itself	Only three fields are editable
`def clean_<field>(self)`	Runs for that field after basic checks	Must return the value
`Decimal('0') < rate <= Decimal('1')`	A chained comparison	Both ends in one line

Check yourself
CommissionRunForm with month '08-2026' is invalid, with 'Use the format YYYY-MM' in form.errors['month'].

Step 4. run_list and the GET side of run_create

What I was thinking
"The list needs the runs and each run's author. select_related fetches the authors in the same query, instead of one query per row."
"A GET to run_create just shows an empty form, opened on last month with the usual rate."

```
@login_required
def run_list(request):
    """All past runs, newest first."""
    runs = CommissionRun.objects.select_related("created_by")[:50]
    return render(request, "commission/run_list.html", {"runs": runs})


@login_required
def run_create(request):
    if request.method == "POST":
        ...                                                    # Step 5
    else:
        form = CommissionRunForm(initial={"month": previous_month(), "rate": Decimal("0.05")})
    return render(request, "commission/run_form.html", {"form": form})
```

Tool	What it does	Why here
`@login_required`	Sends anonymous users to LOGIN_URL	Every report view has it
`Model.objects.select_related('created_by')`	Joins the user table in one query	50 rows, 1 query, not 51
`[:50]`	LIMIT 50 in SQL	Querysets are lazy; slicing limits the query
`request.method == 'POST'`	Was the form submitted?	One URL serves both show and submit
`Form(initial={...})`	Pre-filled values, not yet data	The form opens on last month
`render(request, template, context)`	Fills a template	Returns the HTML response

Check yourself
Open /commission/new/ in November: the month box shows October.

Step 5. The POST side: validate, build, catch

What I was thinking
"Bind the form to what was sent, including files. If it's invalid, fall through to render, and the same template shows the errors."
"save(commit=False) gives me the model object without writing it, so I can fill created_by and the counts first."
"services.py raises ValueError with messages meant for users. Catch only that, and show it at the top of the form."
"read_brokers read the upload to its end; Django must read it again to store it, so rewind with seek(0)."

```
        form = CommissionRunForm(request.POST, request.FILES)
        if form.is_valid():
            run = form.save(commit=False)                    # a model object, not saved yet
            run.created_by = request.user
            upload = form.cleaned_data["broker_file"]
            try:
                brokers = services.read_brokers(upload)
                upload.seek(0)                               # rewind so Django can save the file too
                start, end = month_range(run.month)
                led, notes = get_ledger_slice(start, end,
                                              clients=brokers["Referred User Client ID"],
                                              types=services.TRADING_TYPES)
                report = services.build_commission(led, brokers, run.month, float(run.rate))
                content = excel.build_workbook(report["brokers"], report["clients"])
            except ValueError as err:                        # our own clear messages from services.py
                form.add_error(None, str(err))
                return render(request, "commission/run_form.html", {"form": form})
```

Tool	What it does	Why here
`Form(request.POST, request.FILES)`	A bound form: data + uploads	Without FILES the upload is missing
`form.is_valid()`	Runs every check, fills cleaned_data	False keeps the errors on the form
`form.save(commit=False)`	Builds the object, doesn't save	Add fields the user doesn't fill
`request.user`	The logged-in user	Stored as created_by
`upload.seek(0)`	Back to the start of the file	Django saves the whole file, not an empty tail
`float(run.rate)`	Decimal to float for pandas	The calculation uses floats
`except ValueError as err`	Only expected, user-facing errors	Real bugs still show as errors
`form.add_error(None, msg)`	A form-wide error (not one field)	Shown by {{ form.non_field_errors }}

Check yourself
Upload a CSV without 'Referred User Client ID': the form comes back with 'missing columns' at the top, and no run is saved.

Step 6. Saving and redirecting

What I was thinking
"Copy the headline numbers onto the run; Decimal(str(x)) turns the float into an exact decimal without the float's noise."
"Save the run first (it gets created_at, needed by result_path), then the Excel."
"Redirect after success: refreshing the results page then repeats a harmless GET, never the POST."

```
            summary = report["summary"]
            run.brokers_count = summary["brokers"]
            run.clients_count = summary["clients"]
            run.loss_brokers = summary["loss_brokers"]
            run.total_brokerage = Decimal(str(summary["total_brokerage"]))
            run.save()                                          # saves the row and the uploaded CSV
            run.result_file.save(f"Broker_Commission_{run.month}.xlsx", ContentFile(content))
            for note in notes:
                messages.warning(request, note)
            if led.empty:
                messages.warning(request, f"No BILL or NBP rows were found for these clients in {run.month}; "
                                          "check the month and the ledger paths.")
            messages.success(request, f"Report for {run.month} is ready.")
            return redirect("commission:run_detail", pk=run.pk)
```

Tool	What it does	Why here
`Decimal(str(9784.6))`	Decimal('9784.6')	Decimal(9784.6) would be 9784.600000000000363...
`run.save()`	INSERT the row; store the uploaded file	created_at is set now
`field_file.save(name, ContentFile(bytes))`	Stores bytes as a file and saves the run	The Excel lands under media/
`messages.warning / success`	One-time banners for the next page	Notes from the ledger, and 'ready'
`led.empty`	True if the DataFrame has no rows	A wrong month or path gets a hint
`redirect('app:name', pk=...)`	A 302 to the URL with that name	Post / redirect / get

Check yourself
After generating a report, refresh the page: no second run appears in the list.

Step 7. Detail and download

What I was thinking
"Both take the run's id from the URL. A wrong id should show a 404 page, not crash."
"The preview reads the saved Excel; the band row means the header is on Excel row 2."

```
@login_required
def run_detail(request, pk):
    """Headline numbers plus the first 15 brokers, read back from the saved Excel."""
    run = get_object_or_404(CommissionRun, pk=pk)
    preview = sheet_preview(run.result_file, "Brokers", header_row=2)
    return render(request, "commission/run_detail.html", {"run": run, "preview": preview})


@login_required
def run_download(request, pk):
    run = get_object_or_404(CommissionRun, pk=pk)
    return download(run.result_file, f"Broker_Commission_{run.month}.xlsx")
```

Tool	What it does	Why here
`def run_detail(request, pk)`	pk arrives from in the URL	Already an int
`get_object_or_404(Model, pk=pk)`	The object, or a 404 response	No DoesNotExist crash
`sheet_preview(..., header_row=2)`	HTML table of the first rows	Banded sheet: header on row 2
`{{ preview|safe }}` in the template	Marks the HTML as trusted	Our own to_html output, not user text

Check yourself
Open /commission/99999/: you get the 404 page.

Step 8. URLs, admin and settings

What I was thinking
"The app's urls.py lists its four pages and gives them names; app_name makes the names unique across apps."
"The project's urls.py only includes each app under a prefix. In development it also serves media files."
"The admin needs two lines to list runs; list_display and list_filter make it useful."
"Settings wire it together: the apps, the context processor, media and login, and the ledger."

```
# commission/urls.py
app_name = "commission"
urlpatterns = [
    path("", views.run_list, name="run_list"),
    path("new/", views.run_create, name="run_create"),
    path("<int:pk>/", views.run_detail, name="run_detail"),
    path("<int:pk>/download/", views.run_download, name="run_download"),
]

# nexora_reports/urls.py
urlpatterns = [
    path("", include("core.urls")),
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")),
    path("commission/", include("commission.urls")),
    path("activity/", include("activity.urls")),
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

# commission/admin.py
@admin.register(CommissionRun)
class CommissionRunAdmin(admin.ModelAdmin):
    list_display = ("month", "rate", "brokers_count", "clients_count", "loss_brokers",
                    "total_brokerage", "created_by", "created_at")
    list_filter = ("month",)
    readonly_fields = ("created_at",)

# nexora_reports/settings.py (the parts we added)
INSTALLED_APPS = [..., "core", "commission", "activity"]
TEMPLATES[0]["OPTIONS"]["context_processors"] += ["core.context_processors.report_apps"]
MEDIA_URL, MEDIA_ROOT = "media/", BASE_DIR / "media"
LOGIN_URL, LOGIN_REDIRECT_URL, LOGOUT_REDIRECT_URL = "login", "core:home", "login"
LEDGER_MODULE = os.environ.get("LEDGER_MODULE", "core.demo_ledger")
```

Tool	What it does	Why here
`app_name = 'commission'`	The URL namespace	'commission:run_detail' can't clash with activity's
`path('<int:pk>/', view, name=...)`	A converter: digits only, passed as int	/commission/abc/ is a 404
`include('commission.urls')`	Hands the rest of the URL to the app	The app never knows its prefix
`static(MEDIA_URL, document_root=...)`	Serves uploaded files in development	Only when DEBUG is True
`@admin.register(Model)`	Registers with a custom admin class	Runs appear in /admin/
`list_display, list_filter, readonly_fields`	Columns, filters, read-only fields	Useful lists for free
`'core' before the apps`	Templates are searched in this order	core/base.html is found first
`os.environ.get(name, default)`	A setting from the environment	Switch the ledger without editing code

Check yourself
reverse('commission:run_download', args=[12]) is '/commission/12/download/'.

Part 4. Mistakes the Django layer prevents

If you wrote...	What would happen	What the code does instead
`Form(request.POST) for an upload`	'This field is required' for the file	Form(request.POST, request.FILES)
`<form method='post'> without enctype`	No file arrives	enctype='multipart/form-data' in the template
`render(...) after a successful POST`	Refreshing reruns the report	redirect(...) (post / redirect / get)
`result_file.save(...) before run.save()`	TypeError: created_at is None	run.save() first
`Forgetting upload.seek(0)`	An empty or partial CSV is stored	Rewind after reading
`except Exception: in run_create`	Real bugs shown as form messages	except ValueError only
`CommissionRun.objects.get(pk=pk)`	A server error for a wrong id	get_object_or_404
`FloatField for money`	0.1 + 0.2 != 0.3	DecimalField, Decimal(str(x))
`URL names without app_name`	'run_list' clashes between apps	app_name per app
`A view without @login_required`	Anyone on the network sees commissions	Decorator on every view

Part 5. Checkpoints

Extracted from this guide and run against the project; they pass.

```
import datetime

from django.contrib.auth import get_user_model
from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase
from django.urls import resolve, reverse

from commission import views
from commission.forms import CommissionRunForm
from commission.models import CommissionRun
from core.models import ReportRun, result_path
from core.periods import previous_month


class DjangoLayerCheckpoints(TestCase):
    def test_urls_both_ways(self):
        self.assertEqual(reverse("commission:run_download", args=[12]), "/commission/12/download/")
        match = resolve("/commission/12/")
        self.assertIs(match.func, views.run_detail)
        self.assertEqual(match.kwargs, {"pk": 12})

    def test_model_family(self):
        self.assertTrue(ReportRun._meta.abstract)
        self.assertEqual(CommissionRun._meta.ordering, ["-created_at"])
        run = CommissionRun(month="2026-08")
        run.created_at = datetime.datetime(2026, 11, 5)
        self.assertEqual(result_path(run, "x.xlsx"), "commission/results/2026/11/x.xlsx")

    def test_form_checks(self):
        upload = SimpleUploadedFile("brokers.txt", b"x")
        form = CommissionRunForm(data={"month": "08-2026", "rate": "2"}, files={"broker_file": upload})
        self.assertFalse(form.is_valid())
        self.assertIn("Use the format YYYY-MM", form.errors["month"][0])
        self.assertIn("at most 1", form.errors["rate"][0])
        self.assertIn(".csv", form.errors["broker_file"][0])

    def test_get_shows_last_month(self):
        get_user_model().objects.create_user("analyst", password="pass12345")
        self.client.login(username="analyst", password="pass12345")
        response = self.client.get(reverse("commission:run_create"))
        self.assertEqual(response.context["form"].initial["month"], previous_month())

    def test_wrong_id_is_404(self):
        get_user_model().objects.create_user("analyst", password="pass12345")
        self.client.login(username="analyst", password="pass12345")
        self.assertEqual(self.client.get("/commission/99999/").status_code, 404)
```

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. What does abstract = True do, and what do child models inherit?
2. Why must run.save() happen before run.result_file.save(...)?
3. Why is the month checked with re.fullmatch rather than re.search?
4. What does form.save(commit=False) give you, and why is it needed here?
5. Why does the view call upload.seek(0)?
6. Why redirect after a successful POST?
7. What does app_name = 'commission' change?
8. Why Decimal(str(x)) instead of Decimal(x)?

Answers

1. No table for the base; each child gets the base's fields and its Meta (like ordering) in its own table.
2. result_path uses created_at, which is set by the first save; before that it's None and formatting it fails.
3. fullmatch requires the whole text to match; search would accept '2026-08' inside a longer string.
4. An unsaved model object; the view fills created_by and the headline numbers before saving.
5. read_brokers read the upload to its end; without rewinding, Django would store an empty or partial file.
6. So refreshing the result page repeats a harmless GET, not the report-building POST.
7. URL names become 'commission:...', so two apps can both have a 'run_list' without clashing.
8. str(x) gives the short decimal text the float stands for; Decimal(x) copies the float's binary error.

Appendix. The files, with line numbers

core/models.py

```
  1  from django.conf import settings
  2  from django.db import models
  3
  4
  5  def result_path(instance, filename):
  6      """Each app's results go in their own folder: media/<app>/results/2026/11/<file>."""
  7      return f"{instance._meta.app_label}/results/{instance.created_at:%Y/%m}/{filename}"
  8
  9
 10  def input_path(instance, filename):
 11      return f"{instance._meta.app_label}/inputs/{filename}"
 12
 13
 14  class ReportRun(models.Model):
 15      """Fields every saved report has. Abstract: no table of its own; each app's run model adds its fields."""
 16
 17      month = models.CharField(max_length=7, help_text="YYYY-MM, e.g. 2026-08")
 18      result_file = models.FileField(upload_to=result_path, blank=True)
 19      created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
 20                                     on_delete=models.SET_NULL, related_name="+")
 21      created_at = models.DateTimeField(auto_now_add=True)
 22
 23      class Meta:
 24          abstract = True
 25          ordering = ["-created_at"]
```

Lines	Block	Purpose
5-7	`result_path()`	Results folder per app and month
10-11	`input_path()`	Uploads folder per app
14-25	`ReportRun`	Abstract base model

commission/models.py

```
  1  from decimal import Decimal
  2
  3  from django.db import models
  4
  5  from core.models import ReportRun, input_path
  6
  7
  8  class CommissionRun(ReportRun):
  9      """One broker commission report. month, result_file, created_by and created_at come from ReportRun."""
 10
 11      rate = models.DecimalField(max_digits=5, decimal_places=4, default=Decimal("0.05"))
 12      broker_file = models.FileField(upload_to=input_path)
 13
 14      brokers_count = models.PositiveIntegerField(default=0)
 15      clients_count = models.PositiveIntegerField(default=0)
 16      loss_brokers = models.PositiveIntegerField(default=0)
 17      total_brokerage = models.DecimalField(max_digits=14, decimal_places=2, default=Decimal("0"))
 18
 19      def __str__(self):
 20          return f"Commission {self.month} ({self.created_at:%Y-%m-%d %H:%M})"
```

Lines	Block	Purpose
8-20	`CommissionRun`	The app's own fields

core/forms.py

```python
"""Checks every report form needs."""
import re

from django import forms

MAX_UPLOAD_MB = 5


def clean_month_value(month: str) -> str:
    if not re.fullmatch(r"\d{4}-(0[1-9]|1[0-2])", month or ""):
        raise forms.ValidationError("Use the format YYYY-MM, e.g. 2026-08.")
    return month


def clean_csv_upload(f, max_mb: int = MAX_UPLOAD_MB):
    if not f.name.lower().endswith(".csv"):
        raise forms.ValidationError("Upload the file as a .csv.")
    if f.size > max_mb * 1024 * 1024:
        raise forms.ValidationError(f"The file is larger than {max_mb} MB.")
    return f


MONTH_WIDGET = forms.TextInput(attrs={"type": "month"})       # the browser's month picker
```

Lines	Block	Purpose
9-12	`clean_month_value()`	YYYY-MM check
15-20	`clean_csv_upload()`	.csv and size check
23	`MONTH_WIDGET`	The month picker

commission/forms.py

```python
from decimal import Decimal

from django import forms

from core.forms import MONTH_WIDGET, clean_csv_upload, clean_month_value

from .models import CommissionRun


class CommissionRunForm(forms.ModelForm):
    class Meta:
        model = CommissionRun
        fields = ["month", "rate", "broker_file"]
        widgets = {
            "month": MONTH_WIDGET,
            "rate": forms.NumberInput(attrs={"step": "0.005", "min": "0", "max": "1"}),
        }
        labels = {"rate": "Commission rate (0.05 = 5%)", "broker_file": "Broker referrals CSV"}

    def clean_month(self):
        return clean_month_value(self.cleaned_data["month"])

    def clean_rate(self):
        rate = self.cleaned_data["rate"]
        if not Decimal("0") < rate <= Decimal("1"):
            raise forms.ValidationError("The rate must be above 0 and at most 1 (0.05 means 5%).")
        return rate

    def clean_broker_file(self):
        return clean_csv_upload(self.cleaned_data["broker_file"])
```

Lines	Block	Purpose
10-18	`Meta`	Fields, widgets, labels
20-30	`clean_*()`	Per-field checks

commission/views.py

```python
from decimal import Decimal

from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.core.files.base import ContentFile
from django.shortcuts import get_object_or_404, redirect, render

from core.ledger import get_ledger_slice
from core.periods import month_range, previous_month
from core.web import download, sheet_preview

from . import excel, services
from .forms import CommissionRunForm
from .models import CommissionRun


@login_required
def run_list(request):
    """All past runs, newest first."""
    runs = CommissionRun.objects.select_related("created_by")[:50]
    return render(request, "commission/run_list.html", {"runs": runs})


@login_required
def run_create(request):
    """GET: show the form. POST: validate, calculate, save the run and its Excel, then redirect."""
    if request.method == "POST":
        form = CommissionRunForm(request.POST, request.FILES)
        if form.is_valid():
            run = form.save(commit=False)                      # a model object, not saved yet
            run.created_by = request.user
            upload = form.cleaned_data["broker_file"]
            try:
                brokers = services.read_brokers(upload)
                upload.seek(0)                                 # rewind so Django can save the file too
                start, end = month_range(run.month)
                led, notes = get_ledger_slice(start, end,      # only this month, these clients, these types
                                              clients=brokers["Referred User Client ID"],
                                              types=services.TRADING_TYPES)
                report = services.build_commission(led, brokers, run.month, float(run.rate))
                content = excel.build_workbook(report["brokers"], report["clients"])
            except ValueError as err:                          # our own clear messages from services.py
                form.add_error(None, str(err))
                return render(request, "commission/run_form.html", {"form": form})

            summary = report["summary"]
            run.brokers_count = summary["brokers"]
            run.clients_count = summary["clients"]
            run.loss_brokers = summary["loss_brokers"]
            run.total_brokerage = Decimal(str(summary["total_brokerage"]))
            run.save()                                         # saves the row and the uploaded CSV
            run.result_file.save(f"Broker_Commission_{run.month}.xlsx", ContentFile(content))
            for note in notes:
                messages.warning(request, note)
            if led.empty:
                messages.warning(request, f"No BILL or NBP rows were found for these clients in {run.month}; "
                                          "check the month and the ledger paths.")
            messages.success(request, f"Report for {run.month} is ready.")
            return redirect("commission:run_detail", pk=run.pk)
    else:
        form = CommissionRunForm(initial={"month": previous_month(), "rate": Decimal("0.05")})
    return render(request, "commission/run_form.html", {"form": form})


@login_required
def run_detail(request, pk):
    """Headline numbers plus the first 15 brokers, read back from the saved Excel."""
    run = get_object_or_404(CommissionRun, pk=pk)
    preview = sheet_preview(run.result_file, "Brokers", header_row=2)
    return render(request, "commission/run_detail.html", {"run": run, "preview": preview})


@login_required
def run_download(request, pk):
    run = get_object_or_404(CommissionRun, pk=pk)
    return download(run.result_file, f"Broker_Commission_{run.month}.xlsx")
```

Lines	Block	Purpose
18-21	`run_list()`	Past runs
25-42	`run_create(): POST`	Validate, build, catch
46-59	`run_create(): save`	Save, messages, redirect
66-70	`run_detail()`	Numbers + preview
74-76	`run_download()`	The Excel

commission/urls.py

```
 1  from django.urls import path
 2
 3  from . import views
 4
 5  app_name = "commission"                      # lets templates use {% url 'commission:run_list' %}
 6
 7  urlpatterns = [
 8      path("", views.run_list, name="run_list"),
 9      path("new/", views.run_create, name="run_create"),
10      path("<int:pk>/", views.run_detail, name="run_detail"),
11      path("<int:pk>/download/", views.run_download, name="run_download"),
12  ]
```

commission/admin.py

```
 1  from django.contrib import admin
 2
 3  from .models import CommissionRun
 4
 5
 6  @admin.register(CommissionRun)
 7  class CommissionRunAdmin(admin.ModelAdmin):
 8      list_display = ("month", "rate", "brokers_count", "clients_count", "loss_brokers",
 9                      "total_brokerage", "created_by", "created_at")
10      list_filter = ("month",)
11      readonly_fields = ("created_at",)
```

nexora_reports/urls.py

```
 1  from django.conf import settings
 2  from django.conf.urls.static import static
 3  from django.contrib import admin
 4  from django.urls import include, path
 5
 6  urlpatterns = [
 7      path("", include("core.urls")),                        # home page: every report app
 8      path("admin/", admin.site.urls),
 9      path("accounts/", include("django.contrib.auth.urls")), # login and logout pages
10      path("commission/", include("commission.urls")),
11      path("activity/", include("activity.urls")),
12  ]
13
14  if settings.DEBUG:                                         # serve uploaded and generated files locally
15      urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```